BSc (Hons) in **Computer Science**

SE3062 : Intelligent Systems

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 02

> **Objective & Lecture Mapping:** In Lecture 03, we learned that a mathematically perfect "Table-Driven Agent" is impossible to build due to combinatorial explosion. Instead, we must write Agent Programs. Today, you will implement a **Simple Reflex Agent** using Condition-Action rules, observe its fatal flaws in a partially observable environment, and then upgrade it to a **Model-Based Agent** that maintains an internal memory state.

Part 1: Practical Implementation — Coding the Architectures

Step 1.1: Setting the Trap (Partial Observability)

> To see why Simple Reflex agents fail, we must handicap your agent's sensors. We are moving from a Fully Observable world to a Partially Observable one.

1. Open your `visual_grid_game.py` from Lab 01.
2. Locate the `get_percept(self)` method.
3. Modify it so it **no longer returns** the exact global coordinates (`agent_pos`). Instead, it should only return local booleans, e.g., `{'wall_ahead': True/False, 'food_here': True/False}` based on checking the adjacent cell in its current facing direction.

Step 1.2: The Simple Reflex Agent (Implementation & Failure)

1. Create a new class called `SimpleReflexAgent` .
2. Implement a `sense_and_act(self, percept)` method that uses strictly IF-THEN logic (Condition-Action rules). *Do not use an* `__init__` *method to store history.*

Example Logic: `IF food_here THEN suck; IF wall_ahead THEN turn_left; ELSE move_forward`

3. Run the simulation. **Observe:** Watch the agent get trapped in a corner or a U-shaped wall, infinitely repeating a cycle (e.g., turn left, step forward, hit wall, turn left...).

Step 1.3: The Model-Based Agent (Memory & State)

1. Create a new class called `ModelBasedAgent`.
2. Implement an `__init__(self)` method to initialize an internal memory state (e.g., `self.visited_cells = set()` or a relative movement tracker).
3. In `sense_and_act(self, percept)`, first **update the state** (Transition & Sensor Model) by recording the current percept and the last action taken.
4. Update your IF-THEN rules to query the memory.
   Example Logic: `IF wall_ahead AND left_is_visited THEN turn_right`
5. Run the simulation. **Observe:** The agent should now remember where it has been, realize it is in a loop, and choose an alternate path to escape.

Part 2: Theoretical Evaluation & Lecture Mapping

> Based on your practical observations above, answer the following questions to map your code directly to the architectural theories covered in Lecture 02.

1. (Remember) According to Lecture 02, why is it impossible to program a mathematically perfect "Table-Driven Agent" for complex environments like Chess? What happens as the agent's lifetime increases?

A Table-Driven Agent would need a table containing an action for every possible percept sequence it could ever experience. In complex environments such as Chess, the number of possible situations and percept histories is extremely large.
As the agent's lifetime increases, the number of possible percept sequences increases enormously because every new percept can combine with all previous percepts. This causes a combinatorial explosion, making the table impossible to store and manage in practice.
So instead of storing every possible situation, we use an Agent Program that calculates an appropriate action from the current percept and internal state.

2. (Understand) Look at the code you wrote for your `SimpleReflexAgent`. Identify and explain the specific lines of code that represent the "Condition-Action Rules" discussed in the lecture.

The first rule is:
```
if percept['food_here']:
    return 'Right'
```
This represents: IF food is detected -> THEN move Right.

The second rule is:
```
if percept['wall_ahead']:
    return 'Left'
```
This represents: IF there is a wall ahead -> THEN move Left.

Finally:
```
return 'Right'
```
This represents the default rule: IF no other condition is true -> THEN move Right.

These are called condition-action rules because the agent checks the current percept and immediately chooses an action without using previous experiences or memory.

3. (Analyze) Your `SimpleReflexAgent` likely got stuck in an infinite loop during Step 1.2. Based on the lecture, analyze exactly why this happened. How did the combination of "Partial Observability" and a lack of "Percept History" cause this failure?

The SimpleReflexAgent got stuck because it makes decisions based only on the current percept and does not maintain any memory of previous situations or actions. For example, when it detects a wall, the agent always executes;

```
if percept['wall_ahead']:
    return 'Left'
```

This means that whenever the same percept occurs, the agent will make the same decision again.
The problem becomes worse because the environment is partially observable. The agent cannot see the entire grid or know its exact position. It only receives local information such as whether there is a wall or food in the direction it is facing. Therefore, two different locations in the grid can produce the same percept.
Since the agent has no percept history, it cannot remember that it has already visited a particular area or performed the same action before. It therefore cannot recognize that it is repeating a cycle. For example, it may move left, then encounter another wall and move left again, eventually returning to a situation that looks identical to an earlier one and repeating the same decisions.

4. (Evaluate) In Step 1.3, you added an internal state to your `ModelBasedAgent`. Evaluate how your specific code handles the "Transition Model" (how the world evolves) and the "Sensor Model" (how the agent's actions affect the world).

My ModelBasedAgent handles the Sensor Model by converting the current percept into an internal state:

```
current_state = (
    percept.get('wall_ahead', False),
    percept.get('food_here', False),
    percept.get('toxin_ahead', False)
)
```

For example, it records whether there is a wall, food, or toxin ahead. This is the agent's belief about its immediate situation, built only from what it can actually sense, without knowing its true position on the grid.

The Transition Model is represented by remembering previous state-action pairs:

```
self.visited_states.add((previous_state, self.last_action))
```

For example, if Left has already been tried in the same state, the agent checks:

```
if (current_state, candidate) not in self.visited_states:
    action = candidate
```

and chooses another action. This helps the agent avoid repeating the same action and escape loops.

However, this is a simple reactive model, not a fully predictive one. It remembers what has already been tried rather than predicting the next state before acting. Also, because the internal state only contains the three percept values and not the agent's actual position, two different locations with the same percept are treated as the same state. Therefore, it works well for the small trap in this practical, but it may not work reliably in a larger or more complex maze.

Finalize and Lock Lab 02 Documentation